

Programming Basics 1



01: Variablen & Datentypen

Sommersemester 2025

Simon Dewert

HAW Hamburg

Fakultät Design, Medien und Information

E52, Finkenau 35, 22081 Hamburg

simon.dewert@haw-hamburg.de

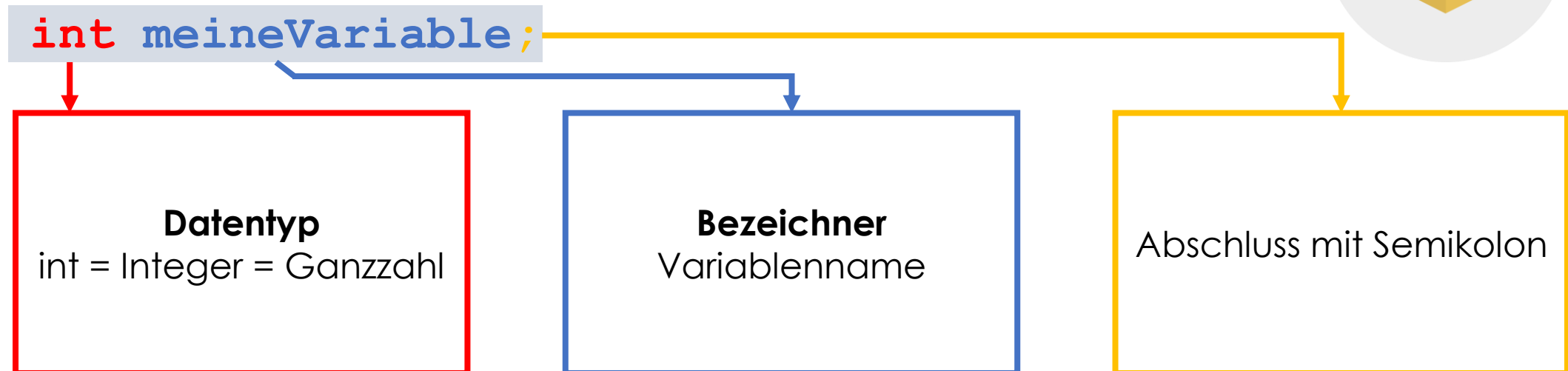
Übersicht dieser Lehreinheit

1. Variablen
2. Datentypen
3. Eingabe – Verarbeitung – Ausgabe
4. Konstanten
5. Casting
6. Wrapper
7. Strings

Variablen

Definition: Eine Variable ist ein Name für einen Speicherbereich, in dem ein Datenobjekt abgelegt werden kann (z. Bsp.: Ganzzahl, ein String etc.)

Deklaration:



Variablen - Bezeichner



Erlaubt

- Buchstaben von **A bis Z** in **Groß- und Kleinschreibung**
- Ziffern von **0 bis 9** (Ausnahme: das erste Zeichen)
- Unterstrich **_** und Dollarzeichen **\$**

Nicht erlaubt

- **Leerzeichen, Punkte** und sonstige **Sonderzeichen**
- Java-**Schlüsselwörter** (siehe nächste Folie)

Konventionen

- Keine Umlaute verwenden
- Mit Kleinbuchstaben beginnen
- Folgewörter groß beginnen („Camel Case“, bsp: `meinWert`)
- Möglichst **kurze** und **aussagekräftige Namen**.

Java-Schlüsselwörter



Diese Wörter dürfen **nicht** als Bezeichner verwendet werden:

abstract	default	if	private	this
assert	do	implements	protected	throw
boolean	double	import	public	throws
break	else	instanceof	return	transient
byte	enum	int	short	try
case	extends	interface	static	void
catch	final	long	strictfp	volatile
char	finally	native	super	while
class	float	new	switch	
continue	for	package	synchronized	

Zwischenfrage



Welche der folgenden Bezeichner sind erlaubt?

`$tring`

`_blaBliBlub`

`hamburg-meine-perle`

`3fragezeichen`

`j4v4`

Variablen - Datentypen

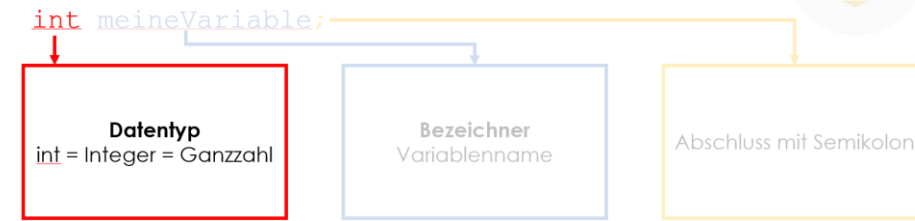


- Jede Variable hat einen Datentyp.
- Der Typ legt fest, welche Art von Daten in der Variable gespeichert werden können.
- Es wird zwischen **elementaren/primitiven** und **komplexen** Datentypen unterschieden.
- Der Datentyp liefert Informationen für den Compiler:
 - *Wieviele Speicher ist zu reservieren?*
 - *Welche Operationen sind möglich?*
 - *Wie groß ist der gültige Wertebereich?*



elementare Datentypen

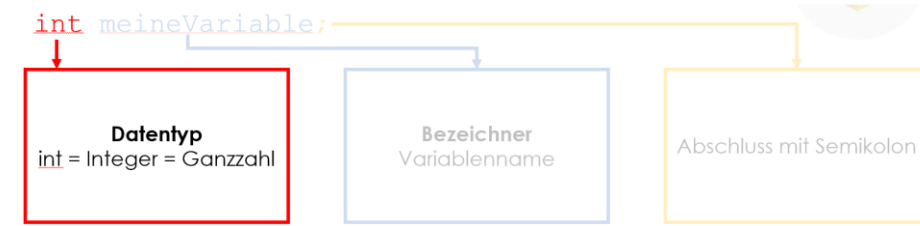
Variablen



	Typ	Größe (Bits)	Werte-bereich / Genauigkeit	Beispiel	Standardwert
Wahrheitswert	boolean	8	false, true	boolean isPositiv = true;	false
Ganzzahlen negativ & positiv	byte	8	-128 bis 127	byte numberOfStudents = 60;	0
	short	16	-32768 bis 32767	short cologne = 4711;	0
	int	32	$-2^{31} \dots (2^{31}-1)$	int negativeNumber = -1312;	0
	long	64	$-2^{63} \dots (2^{63}-1)$	long serialNumber = 9223372017L;	0
Fließkommazahlen	float	32	8 Stellen (Vor- und Nachkomma)	float average = 0.0045f; float scientific = 3.0e8f;	0.0f
	double	64	16 Stellen (Vor- und Nachkomma)	double epsilon = 1e-6; double oneOneThird = 1.3333;	0.0
Zeichen	char	16	Alle Unicode-Zeichen	char atCharacter = '@';	\u0000

komplexe Datentypen

Variablen



- Ein komplexer Datentyp ist aus mehreren primitiven Datentypen zusammen gesetzt.
- Komplexe Datentypen können vom Nutzer selbst, dem Bedarf entsprechend, angelegt werden.

Beispiele: (Klassen, Interfaces, Strings, Arrays u.v.m.)

Datentyp (Klasse)	Beschreibung
String	Text (auch: Zeichenkette)
Date	Datum und Uhrzeit
BigDecimal	Beliebig große Dezimalzahlen
List	Listen

Variablen – Wertzuweisung

Zu Beginn (Initialisierung):

```
int variableEins = 1000;  
int variableZwei = 2000, variableDrei = 3000;
```

Datentyp

Zuweisungs-Operator

Sog. „Literal“

Zur Laufzeit:

```
variableZwei = variableDrei;    // variableZwei: 3000  
variableEins = variableZwei;    // variableEins: 3000
```

Ausgabe (zur Laufzeit):

```
System.out.println(variableEins);
```

Ausgabe: 3000

Begriffsklärung

Initialisierung:

Zuweisung eines Initial- oder Anfangswertes zu einer Variablen (oder Objekt).

Laufzeit:

Die Zeit, in der ein Programm ausgeführt wird.

An die Arbeit!



Arbeit!
Arbeit!

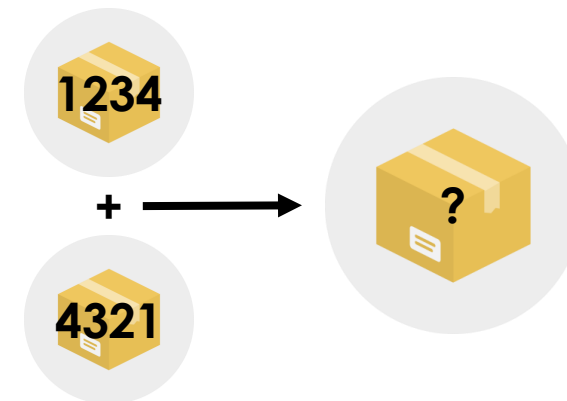
Aufgabe 1.1



Legen Sie zunächst ein **neues Java-Projekt** namens **vorlesung_01** an!

Erstellen Sie dann in diesem neuen Projekt ein Java-Programm (Klasse) namens **AdditionsProgramm**, das die Summe aus zwei Ganzzahl-Variablen mit den initial zugeordneten Werten **1234** und **4321** in einer **dritten Variable aufsummiert** und den Inhalt dieser Variable auf der Konsole ausgibt!

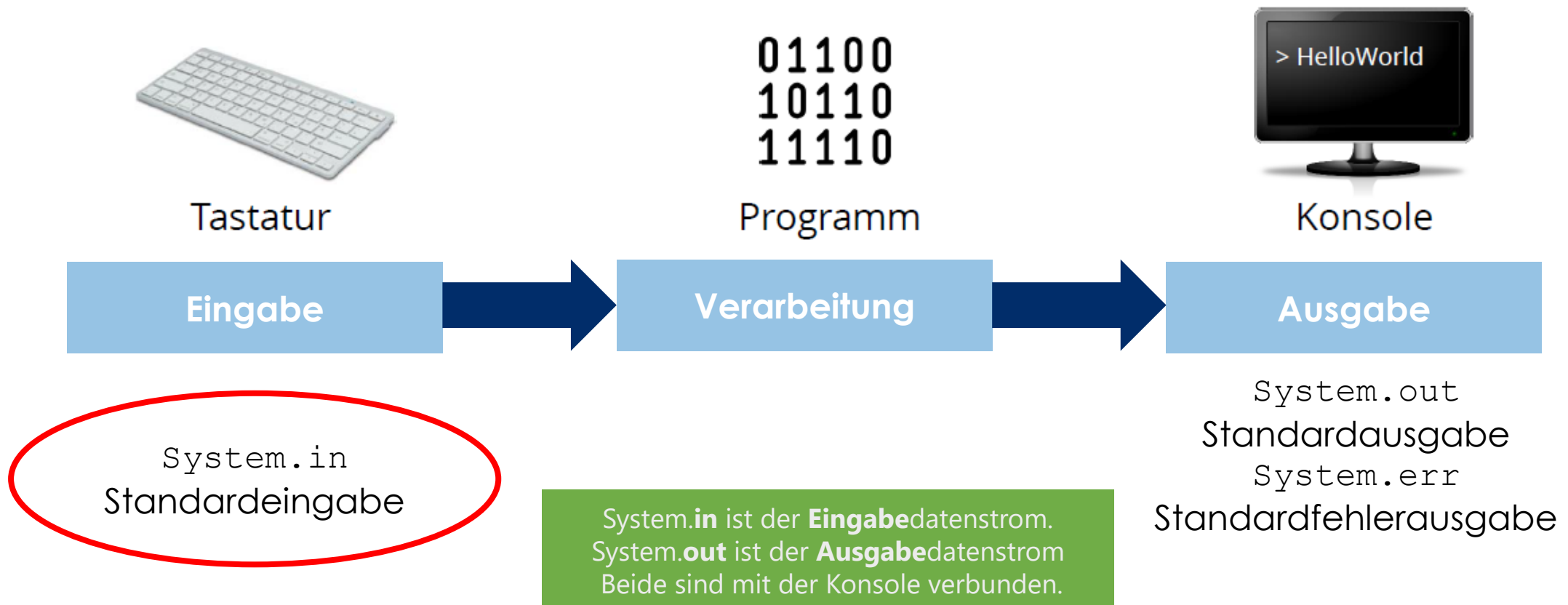
Wie lautet das Ergebnis dieser Addition?



„main“ Methode generieren: main eintippen + Enter
Konsolenausgabe: siehe HelloWorld.java
sout eintippen + Enter

Eingabe – Verarbeitung – Ausgabe

Grundsätzliches Prinzip vieler Computerprogramme:





Eingabe über `System.in`

Seit Java5: Klasse „**Scanner**“ zum Einlesen von Daten:

```
import java.util.Scanner;
```

Klassen, die nicht zum Paket ("package") `java.lang` gehören, müssen importiert werden!

[...]

Vorgriff Objekte

```
Scanner meinScanner = new Scanner(System.in); // Übergabe Standardeingabe  
int meineZahl = meinScanner.nextInt();       // Einlesen eines Integers
```

[...]

```
meinScanner.close();
```

Das Scanner-Objekt wird geschlossen.



Ausgabe über `System.out`

1. Methode `print` → Ausgabe mit englischer Formatierung:

```
System.out.print("2.3");  
System.out.print("2.3");
```

Ausgabe:
2.32.3

2. Methode `println` → Ausgabe mit englischer Formatierung und Zeilenumbruch am Ende:

```
System.out.println("2.3");  
System.out.println("2.3");
```

Ausgabe:
2.3
2.3

IntelliJ-Kurzschreibweise: **sout** → **Enter**

Aufgabe 1.2



Erstellen Sie im Projekt **vorlesung_01** ein Java-Programm namens **AdditionsProrgammEVA**, das zwei ganze Zahlen, die nacheinander eingegeben werden, addiert!

Versuchen Sie dieselbe Ausgabeform wie im Beispiel zu bekommen.

Beispiel:

Geben Sie die erste Zahl ein: 12

Eingabe

Geben Sie die zweite Zahl ein: 4

Eingabe

Die Summe der beiden Zahlen ist 16

Ausgabe

Ganzzahl-Literale

Variablen der Ganzzahl-Datentypen (**byte**, **short**, **int** und **long**) können mit vier unterschiedlichen Literalen „gefüttert“ werden.

Als Beispiel hier eine Variable des Datentyps `int` (es könnte aber auch `byte`, `short` oder `long` sein)

Literale

```
int dezimalzahl = 5;  
int binärzahl = 0b00000010;    // dezimal: 2  
int oktalzahl = 017;           // dezimal: 15  
int hexadezimalzahl = 0x17;    // dezimal: 23
```

Konstanten

Eine Konstante bezeichnet einen festen Wert, der sich während der Laufzeit des Programms **nicht ändern** soll.

Anwendungsbeispiele:

- Maximale Anzahl einer bestimmten Menge
- Konstante mathematische Größen (Pi etc.)

Konstanten werden in Java durch das Schlüsselwort **final** definiert und werden üblicherweise in GROSSBUCHSTABEN geschrieben;

Einzelworte werden durch Unterstrich getrennt:

```
final int MAXIMALE_ANZAHL = 100;
```

Der Wert einer Konstanten kann nachträglich nicht mehr verändert werden!

Implizites Casting (Widening)

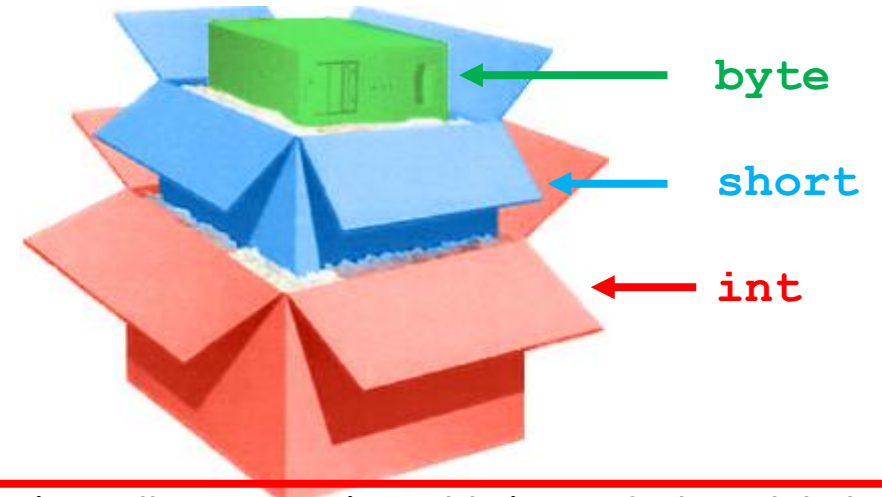
Wenn der Wert einer Variable in eine zweite Variable anderen Datentyps übernommen (kopiert) werden soll, so wird der Wert implizit (automatisch) konvertiert, wenn folgende Gesetzmäßigkeit beachtet wird:

byte < short < int < long < float < double

```
byte meinByte      = 65;  
short meinShort    = meinByte;  
int meinInt        = meinShort;  
long meinLong      = meinInt;  
float meinFloat    = meinLong;  
double meinDouble  = meinFloat;
```

zulässig

```
meinByte = meinDouble; nicht zulässig
```



Prinzip: Alles aus einer kleinen Schachtel, passt **immer** auch in eine größere

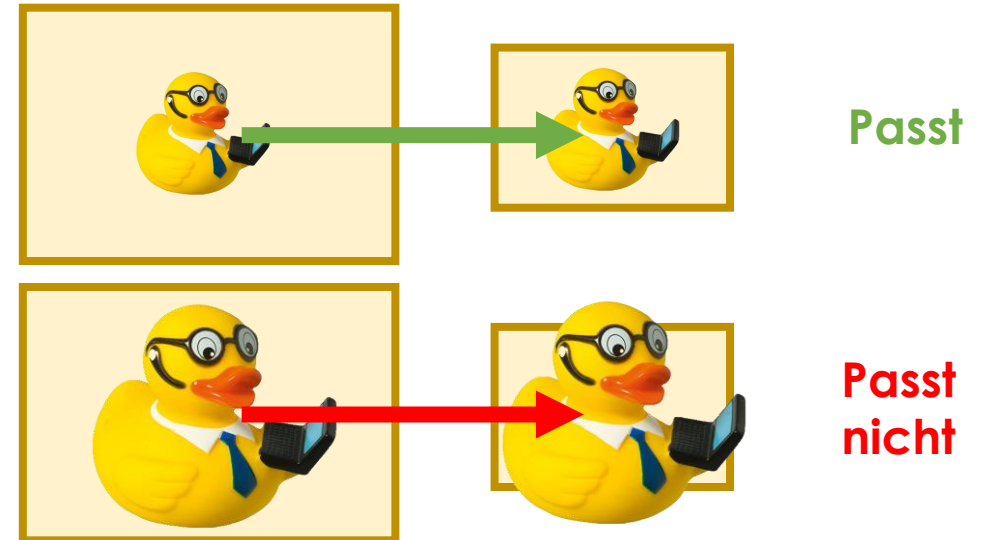
Explizites Casting (Narrowing)

Wenn entgegen der Reihenfolge konvertiert werden soll, so muss der **Datentyp der Zielvariable** explizit in Klammern **vor der Quellvariable** angegeben werden:

```
meinByte = (byte) meinDouble;
```

Beachte:

Der Wert der Quellvariable muss im zulässigen Wertebereich des Datentyps der Zielvariablen liegen!



Prinzip:

In einer großen Schachtel **kann** auch etwas kleines liegen, welches man in eine kleinere Schachtel packen kann

Aufgabe 1.3



Erstellen Sie im Projekt **vorlesung_01** ein Java-Programm namens **DivisionsProgrammEVA**, in welchem zwei einzugebende Fließkommazahlen dividiert werden!

(Optional) Ein- und Ausgabe sollen auf zwei Nachkommastellen genau sein!

Beispiel:

Geben Sie die erste Fließkommazahl ein: 5,3 Eingabe

Geben Sie die zweite Fließkommazahl ein: 4,6 Eingabe

Die Division der beiden Zahlen ergibt 1,15 Ausgabe

Wrapper-Klassen

Zu jedem elementaren Datentyp gibt es eine entsprechende Klasse, die sogenannte Wrapper-Klasse, die nützliche Methoden beinhaltet. Außerdem ermöglichen Wrapper die Werte als Referenz-Parameter zu verwenden.

Beispiel:

```
int meinInteger = 20;
Integer meinWrapper = Integer.valueOf(meinInteger);

Integer.MAX_VALUE;
Integer.max(a, b);
```

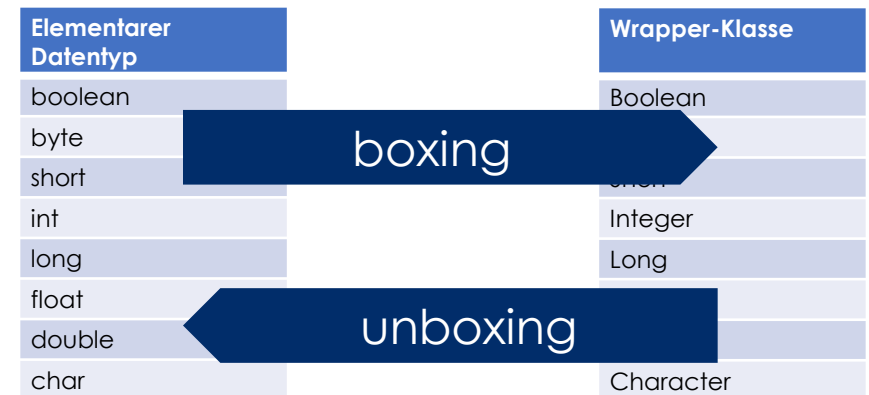
Elementarer Datentyp	Wrapper-Klasse
boolean	Boolean
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character

Wrapper-Klassen → **komplexe** Datentypen

Autoboxing & Autounboxing

Autoboxing = Automatische Umwandlung eines elementaren Datentyps in seine Wrapper-Klasse.

Autounboxing = Automatische Umwandlung einer Wrapper-Klasse in seinen elementaren Datentyp.



Beispiel:

```
int meinInteger = 10;  
Integer meinWrapper = meinInteger; // Autoboxing  
meinInteger = meinWrapper; // Autounboxing
```

Strings



Die **Klasse** String dient der Verarbeitung von **Zeichenketten**.

„Offizielle“ Instanziierung eines Strings: `String meinString = new String("Mein Text");`

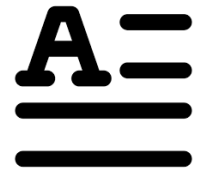
Von der Handhabung her verhält sich die Klasse String wie ein elementarer Datentyp bzw. wie Wrapper-Klasse für Zeichenketten (mit Autoboxing):

```
String meinString = "MeinText";           // Literal-Initialisierung
String meinString2 = "MeinText" + "ist vielsagend"; // Additions-Operator
```

Wie eine Wrapper-Klasse hat auch die Klasse String nützliche Methoden:

```
System.out.println(meinString.equals("MeinText")); //Ausgabe: true
```

Siehe: <https://docs.oracle.com/javase/10/docs/api/java/lang/String.html>



String - Casting

Als Parameter der Methode `println()` und beim „Hinzuzaddieren“ zu Strings werden elementare Datentypen **implizit** in Strings umgewandelt.

Ansonsten muss explizit durch Wrapper-Methoden umgewandelt werden:

1. Elementaren Datentyp $\rightarrow$ String: `valueOf(x).toString()`

Beispiel:

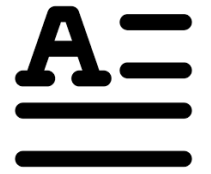
```
int drei = 3;  
String dreiString = Integer.valueOf(drei).toString();
```

2. String $\rightarrow$ Elementarer Datentyp: `parse<Datentyp>(String str)`

Beispiel:

```
String dreiString = "3";  
int drei = Integer.parseInt(dreiString);
```

String - Sonderzeichen



`\n` neue Zeile `https://www.baeldung.com/java-string-newline`

`\t` Tabulator

`\\` einen Backslash darstellen

Aufgabe 1.4



Erstellen Sie im Projekt **vorlesung_01** ein Java-Programm namens **GrussProgramm**, bei dem der Benutzer seinen Vor- und Nachnamen eingibt und mit diesem danach begrüßt wird!

Beispiel:

Geben Sie ihren Vornamen ein: John Eingabe

Geben Sie ihren Nachnamen ein: Doe Eingabe

Willkommen John Doe! Ausgabe

Viele Dank für Ihre Aufmerksamkeit



Hausaufgabe



Erstellen Sie im Projekt **vorlesung_02** ein Java-Programm namens **Grundrechenarten**, in welchem zwei einzugebende Fließkommazahlen addiert, subtrahiert, dividiert und multipliziert werden!

Beispiel:

Geben Sie die erste Zahl ein: 5,3

Eingabe

Geben Sie die zweite Zahl ein: 4,6

Eingabe

$$5,3 + 4,6 = 9,9$$

$$5,3 - 4,6 = 0,7$$

$$5,3 * 4,6 = 24,4$$

$$5,3 / 4,6 = 1,15$$

Ausgabe

Versucht herauszufinden wie sich das Ergebnis auf eine Nachkommastelle formatieren lässt